1/28/2025	9 AM - 10 AM	OOPs Concepts

1. **Abstract Class**
2. **Polymorphism**
3. **Inheritance**
4. **Encaptulation**

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around data, or **objects**, and the operations (methods) that can be performed on them. In OOP, the program is structured around **classes** and **objects** rather than functions and logic. The four main concepts of OOP are:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C# code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

10 AM - 11 AM	Abstraction

1. Abstraction

Abstraction involves hiding complex implementation details and showing only the necessary features of an object. It helps in reducing complexity and allows the programmer to focus on high-level functionality without needing to understand all the internal workings.

- **Abstract classes**: Cannot be instantiated and may contain abstract methods (methods without implementation) that must be implemented by derived classes.
- **Interfaces**: Define a contract that classes must adhere to, without providing implementation details.

Abstraction in object-oriented programming (OOP) refers to the concept of hiding complex implementation details and exposing only the essential features or functionalities of an object. This simplifies interaction with objects by providing a high-level interface and hiding the low-level complexities.

In C#, abstraction is implemented using **abstract classes**, **interfaces**, and **methods** that allow you to define only the essential functionality, while hiding the unnecessary complexity.

Example of Abstraction in C# using an Abstract Class

Let's consider a scenario where we want to model different types of Shape objects (such as Circle, Rectangle, etc.), where each shape has the ability to calculate its area, but the calculation logic differs for each type of shape. We'll use **abstraction** to hide the details of how the area is calculated and only expose the necessary method to compute the area.

Code Example:

```
using System;


// Abstract class - This provides a common interface for all shapes
public abstract class Shape
{
   // Abstract method - it will be implemented by derived classes
   public abstract double CalculateArea();


   // A regular method that is shared by all shapes
   public void DisplayArea()
   {
      Console.WriteLine("The area of the shape is: " + CalculateArea());
   }
}
```

```
// Derived class - Circle
public class Circle : Shape
{
   public double Radius { get; set; }
  //Constructor
   public Circle(double radius)
   {
      Radius = radius;
   }

   // Implementing the abstract method to calculate the area of a circle
   public override double CalculateArea()
   {
      return Math.PI * Radius * Radius;  // Area of a circle = π * r^2
   }
}

// Derived class - Rectangle
public class Rectangle : Shape
{
   public double Width { get; set; }
   public double Height { get; set; }
   //constructor
   public Rectangle(double width, double height)
   {
      Width = width;
```

```
        Height = height;
    }

    // Implementing the abstract method to calculate the area of a rectangle
    public override double CalculateArea()
    {
        return Width * Height;  // Area of a rectangle = width * height
    }
}

class Program
{
    static void Main()
    {
        // Creating instances of derived classes
        Shape circle = new Circle(5);  // Circle with radius 5
        Shape rectangle = new Rectangle(4, 6);  // Rectangle with width 4 and height 6

        // Displaying the area for both shapes
        circle.DisplayArea();  // Output: The area of the shape is: 78.53981633974483
        rectangle.DisplayArea();  // Output: The area of the shape is: 24
    }
}
```

Explanation of Abstraction:

1. **Abstract Class Shape**:
    - The Shape class is **abstract**, meaning it cannot be instantiated directly.
    - It defines an **abstract method** CalculateArea(), which means that derived classes must implement their own version of how to calculate the area of that particular shape.

- It also provides a **regular method** DisplayArea(), which calls CalculateArea() and prints the result. This method is shared by all derived classes, simplifying the code for displaying the area.

2. **Derived Classes (Circle and Rectangle)**:
   - Both Circle and Rectangle inherit from Shape and implement the CalculateArea() method in their own way, based on their specific formulas for calculating area.
   - For example, the Circle class calculates area using the formula πr2\pi r^2πr2, while the Rectangle class uses width×height\text{width} \times \text{height}width×height.
3. **Abstraction in Action**:
   - In the Main method, we can create objects of Circle and Rectangle and use the DisplayArea() method without needing to know the internal details of how each shape calculates its area.
   - This abstracts away the complexity of each shape's calculation and provides a simple interface (DisplayArea()) to interact with them.

Key Points of Abstraction:

- **Hiding Complexity**: The user of the Shape class doesn't need to understand the specific implementation of CalculateArea() for each shape. They only need to know that calling DisplayArea() will give the area of the shape.
- **Flexible Interface**: The Shape class provides a common interface (CalculateArea()), and different types of shapes implement that interface in their own way.
- **Reusability and Maintainability**: Since the area calculation is abstracted in each derived class, the code is easier to maintain. If you need to change the way the area of a Circle is calculated, you only need to update the Circle class, and the rest of the code remains unaffected.

11 AM - 12 PM	Encapsulation

2. Encapsulation

Encapsulation is the concept of bundling the data (fields) and the methods (functions) that operate on the data into a single unit, or class. It also involves restricting direct access to some of the object's components to prevent unintended interference and misuse.

- **Private fields**: Data is hidden within the class and can only be accessed or modified through public methods (getters and setters).
- **Public methods**: These provide controlled access to the private data, ensuring that any changes to the data are validated and secure.

Example 1:

```
public class BankAccount
{
  private decimal balance;  // Private field

  // Public method to deposit money
  public void Deposit(decimal amount)
  {
    if (amount > 0)
    {
      balance += amount;
    }
  }

  // Public method to withdraw money
  public void Withdraw(decimal amount)
  {
```

```
    if (amount > 0 && amount <= balance)
    {
      balance -= amount;
    }
  }

  // Public method to get the balance
  public decimal GetBalance()
  {
    return balance;
  }
}
```

In this example, the balance is **encapsulated** within the class, and the outside world can interact with the balance only through the Deposit, Withdraw, and GetBalance methods.

Example 2:

Let's consider a class Person where we encapsulate the age and name fields. We will provide getter and setter methods to access and update these fields.

```
using System;

public class Person
{
  // Private fields (encapsulated data)
  private string name;
  private int age;
```

```
// Public property to get and set the name
   public string Name
   {
      get { return name; }
      set { name = value; }
   }

   // Public property to get and set the age
   public int Age
   {
      get { return age; }
      set
      {
         if (value >= 0)
         {
            age = value; // Only allow non-negative age
         }
         else
         {
            Console.WriteLine("Age cannot be negative.");
         }
      }
   }

   // Constructor to initialize the person's name and age
   public Person(string name, int age)
   {
```

```
        Name = name;
        Age = age;
    }

    // Method to display person's information
    public void DisplayInfo()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }
}

class Program
{
    static void Main()
    {
        // Creating a Person object
        Person person = new Person("Alice", 30);

        // Accessing properties via getter and setter
        Console.WriteLine("Before update:");
        person.DisplayInfo();  // Output: Name: Alice, Age: 30

        // Updating name and age via setters
        person.Name = "Bob";
        person.Age = 35;

        Console.WriteLine("\nAfter update:");
        person.DisplayInfo();  // Output: Name: Bob, Age: 35
```

```
    // Trying to set a negative age (will be blocked by setter)
    person.Age = -5;  // Output: Age cannot be negative.
  }
}
```

12PM - 1 PM	Practice
1 PM – 2 PM	Lunch Break
2 PM - 3 PM	Inheritance

3. Inheritance

Inheritance allows a class to inherit properties and behaviours (fields and methods) from another class. The **base class** (or parent class) provides common functionality, while the **derived class** (or child class) can extend or override that functionality.

- **Base class**: A class that is inherited by other classes.
- **Derived class**: A class that inherits from another class.

Example 1:

```
public class Animal
{
  public void Eat()
  {
    Console.WriteLine("Eating...");
  }
}


public class Dog : Animal
```

```
{
   public void Bark()
   {
      Console.WriteLine("Barking...");
   }
}
```

Here, the Dog class inherits the Eat method from the Animal class, so you don't need to define it again. A Dog object can call both Eat (inherited) and Bark (its own).

Usage:

```
Dog dog = new Dog();
dog.Eat();  // Inherited method
dog.Bark(); // Derived class method
```

Example 2:

```
using System;

// Base class - Vehicle
public class Vehicle
{
   // Properties of the base class
   public string Make { get; set; }
   public string Model { get; set; }
   public int Year { get; set; }

   // Constructor of the base class
   public Vehicle(string make, string model, int year)
   {
      Make = make;
```

```
        Model = model;
        Year = year;
    }

    // Method in the base class
    public void DisplayInfo()
    {
        Console.WriteLine($"Vehicle Info: {Year} {Make} {Model}");
    }
}

// Derived class - Car (inherits from Vehicle)
public class Car : Vehicle
{
    // Additional property specific to the Car class
    public int NumberOfDoors { get; set; }

    // Constructor of the derived class
    public Car(string make, string model, int year, int numberOfDoors)
        : base(make, model, year)  // Call the base class constructor
    {
        NumberOfDoors = numberOfDoors;
    }

    // Method in the derived class
    public void DisplayCarInfo()
    {
        DisplayInfo();  // Call the inherited method from Vehicle
```

```
        Console.WriteLine($"Number of doors: {NumberOfDoors}");
    }
}

// Another derived class - Truck (inherits from Vehicle)
public class Truck : Vehicle
{
    // Additional property specific to the Truck class
    public double PayloadCapacity { get; set; }

    // Constructor of the derived class
    public Truck(string make, string model, int year, double payloadCapacity)
        : base(make, model, year)  // Call the base class constructor
    {
        PayloadCapacity = payloadCapacity;
    }

    // Method in the derived class
    public void DisplayTruckInfo()
    {
        DisplayInfo();  // Call the inherited method from Vehicle
        Console.WriteLine($"Payload capacity: {PayloadCapacity} kg");
    }
}

class Program
{
    static void Main()
```

```
  {
    // Create instances of the derived classes
    Car myCar = new Car("Toyota", "Camry", 2020, 4);
    Truck myTruck = new Truck("Ford", "F-150", 2022, 1200);

    // Call methods on the derived class objects
    Console.WriteLine("Car Info:");
    myCar.DisplayCarInfo();  // Output: Vehicle Info: 2020 Toyota Camry, Number of doors: 4

    Console.WriteLine("\nTruck Info:");
    myTruck.DisplayTruckInfo();  // Output: Vehicle Info: 2022 Ford F-150, Payload capacity: 1200 kg
  }
}
```

3 PM - 4 PM	Polymorphism

4. Polymorphism

Polymorphism allows an object to take on many forms. It allows methods or objects to behave differently based on the context, enabling you to use a method or object in a more flexible way.

There are two types of polymorphism in C#:

- **Method Overloading**: The ability to define multiple methods with the same name but different parameters.
- **Method Overriding**: The ability of a subclass to provide a specific implementation of a method that is already defined in its superclass.

Example (Method Overriding):

It is an example of **runtime polymorphism**.

```
using System;

// Base class - Animal
public class Animal
{
   // Virtual method in the base class
   public virtual void MakeSound()
   {
      Console.WriteLine("The animal makes a sound.");
   }
}

// Derived class - Dog
public class Dog : Animal
{
   // Overriding the MakeSound method in the derived class
   public override void MakeSound()
   {
      Console.WriteLine("The dog barks.");
   }
}

// Derived class - Cat
public class Cat : Animal
{
   // Overriding the MakeSound method in the derived class
```

```
    public override void MakeSound()
    {
        Console.WriteLine("The cat meows.");
    }
}

class Program
{
    static void Main()
    {
        // Create instances of the derived classes
        Animal myAnimal = new Animal();
        Animal myDog = new Dog();
        Animal myCat = new Cat();

        // Call the MakeSound method for each object
        Console.WriteLine("Animal Sound:");
        myAnimal.MakeSound();  // Output: The animal makes a sound.

        Console.WriteLine("\nDog Sound:");
        myDog.MakeSound();  // Output: The dog barks.

        Console.WriteLine("\nCat Sound:");
        myCat.MakeSound();  // Output: The cat meows.
    }
}
```

Example (Method Overloading):

```
using System;

public class Calculator
{
   // Overloaded method to add two integers
   public int Add(int a, int b)
   {
      return a + b;
   }

   // Overloaded method to add three integers
   public int Add(int a, int b, int c)
   {
      return a + b + c;
   }

   // Overloaded method to add two floating-point numbers
   public double Add(double a, double b)
   {
      return a + b;
   }

   // Overloaded method to add an integer and a floating-point number
   public double Add(int a, double b)
   {
      return a + b;
```

```
    }
}

class Program
{
    static void Main()
    {
        Calculator calc = new Calculator();

        // Calling Add with two integers
        Console.WriteLine("Sum of 10 and 20 (int): " + calc.Add(10, 20));

        // Calling Add with three integers
        Console.WriteLine("Sum of 10, 20, and 30 (int): " + calc.Add(10, 20, 30));

        // Calling Add with two floating-point numbers
        Console.WriteLine("Sum of 10.5 and 20.5 (double): " + calc.Add(10.5, 20.5));

        // Calling Add with an integer and a floating-point number
        Console.WriteLine("Sum of 10 and 20.5 (int + double): " + calc.Add(10, 20.5));
    }
}
```

Evaluation Session
Evaluation Session
HR/All Practice Session

```
Settings
- task: DotNetCoreCLI@2
  inputs:
    command: 'publish'
    publishWebProjects: false
    projects: '$(project)'
    arguments: '--configuration $(BuildConfiguration) --output $(Build.ArtifactStagingDirectory)'
    zipAfterPublish: false
```